5.1 Mini-Calc typing

Types: T ::= num | str.

Typing judgement: $\Gamma \vdash e : T$ where the environment maps variables to types.

Well-formedness of sequence: e_1, e_2 requires e_1 to be either a number or a string, result type is type of e_2.

Typing rules (ASCII inference style):

```
[NUM]        --------------------
             Γ ⊢ 1 : num

[STR]        -----------------------
             Γ ⊢ "Uhhh" : str

[ADD-num]    Γ ⊢ e1 : num    Γ ⊢ e2 : num
             ------------------------------------
             Γ ⊢ e1 + e2 : num

[ADD-str]    Γ ⊢ e1 : str    Γ ⊢ e2 : str
             ------------------------------------
             Γ ⊢ e1 + e2 : str

[STR-coerce] Γ ⊢ e : num
             ---------------
             Γ ⊢ str(e) : str

[ADD-mix-L]  Γ ⊢ e1 : num    Γ ⊢ e2 : str
             ------------------------------------
             Γ ⊢ e1 + e2 : str        (num coerced to str)

[ADD-mix-R]  Γ ⊢ e1 : str    Γ ⊢ e2 : num
             ------------------------------------
             Γ ⊢ e1 + e2 : error      (disallow str + num)

[PRINT]      Γ ⊢ e : str
             ---------------
             Γ ⊢ print(e) : num       (status code)

[SEQ]        Γ ⊢ e1 : T_1    Γ ⊢ e2 : T_2
             ------------------------------------------------
             Γ ⊢ e1 , e2 : T_2
```

Notes:

- Only `num + num` and `str + str` are directly allowed; `num + str` is allowed via coercion on the left operand. `str + num` is rejected (no coercion from str to num).
- `str(e)` converts numbers to strings; applying it to a string is not needed because concatenation already accepts strings.
- `print` consumes a string and yields a numeric status.

5.2 C++: undecidable type checking (template non-termination)

A minimal program that forces template instantiation to diverge during type checking:

```
template<int N>
struct Loop {
    using next = Loop<N + 1>;     // forces another instantiation
    using type = typename next::type;
};

int main() {
    typename Loop<0>::type x;     // triggers infinite instantiation
}
```

Compiling (`g++ -std=c++17 loop.cpp`) makes the compiler instantiate `Loop<0> -> Loop<1> -> ...` until it hits the instantiation-depth limit or never terminates, demonstrating that type checking reduces to an unbounded computation and is therefore undecidable in C++ templates.

5.3 Simply typed λ-calculus: can we type x x?

In STLC without recursive types, there is no context Γ and type T such that Γ ⊢ x x : T.

Reason: to type the application we need Γ ⊢ x : T1 -> T and Γ ⊢ x : T1 for the same variable. This forces T1 = T1 -> T, which has no finite solution in STLC's simple types. Therefore x x is untypable unless the language is extended with recursive types or a fixpoint combinator.